

⚠️ **PRIVATE — DO NOT COMMIT THIS FILE TO GIT** ⚠️

Contains live API keys. Add this file to .gitignore immediately.

JMD ONLINE BOOK — GOD-LEVEL AUTO-BUILD + AUTO-DEPLOY PROMPT

Paste this entire file into: Claude Code / Cursor / Codex / OpenCode CLI

Cost to build: \$0 | Cost to deploy: \$0 | Cost to run: \$0

YOUR CREDENTIALS (pre-filled — do not change)

```
NEXT_PUBLIC_SUPABASE_URL=https://zkvrlwqcfeecsecrzlnu.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdX
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYm
RESEND_API_KEY=re_fHwpnKsC_2wXwZgxfsLHXBmDj36obxj2
NEXT_PUBLIC_APP_NAME=JMD Online Book
OTP_SECRET=d3c8e48db5d4d7f914b055368210266d69deb9b49c745247e1bdce39dbd2f760
```

`NEXT_PUBLIC_APP_URL` will be set automatically after Vercel deploy in Phase 3.

MISSION

You are a senior full-stack engineer. Execute every phase in order without stopping. Build and deploy a complete, production-grade online betting/gaming platform called **JMD Online Book** targeting mobile-first Southeast Asian users.

Non-negotiable constraints:

- Zero paid services. Free tiers only.
 - Every file must be fully written — no stubs, no TODOs.
 - `next build` must pass with zero TypeScript errors before deploying.
 - All API keys come from the credentials block above. Never hardcode elsewhere.
-

FREE STACK

Layer	Service	Free Limit
Framework	Next.js 15 App Router	Unlimited
Database	Supabase PostgreSQL	500 MB
Auth	Supabase Auth	50,000 MAU
Realtime	Supabase Realtime	200 concurrent
File Storage	Supabase Storage	1 GB
Email OTP	Resend	3,000 emails/month
Deployment	Vercel Hobby	Free forever
DNS/SSL	Vercel	Included free

PHASE 1 — PROJECT BOOTSTRAP

1.1 Create project

```
npx create-next-app@latest jmd-online-book \
  --typescript --tailwind --eslint --app \
  --src-dir --import-alias "@/*" --turbo --no-git
```

```
cd jmd-online-book
```

1.2 Install all dependencies

```
npm install \
  @supabase/supabase-js \
  @supabase/ssr \
  zustand \
  zod \
  react-hook-form \
  @hookform/resolvers \
  framer-motion \
  react-hot-toast \
  react-icons \
  recharts \
  date-fns \
  clsx \
  tailwind-merge \
  resend \
  lucide-react \
  @radix-ui/react-dialog \
  @radix-ui/react-tabs \
  @radix-ui/react-select \
  @radix-ui/react-switch \
  @radix-ui/react-dropdown-menu

npm install -D @types/node
```

1.3 Create .env.local

Create file `.env.local` at project root with exactly this content:

```
NEXT_PUBLIC_SUPABASE_URL=https://zkvrlwqcfeecsecrzlnu.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdX
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYm
RESEND_API_KEY=re_fHwpnKsC_2wXwZgxfsLHXBmDj36obxj2
NEXT_PUBLIC_APP_NAME=JMD Online Book
NEXT_PUBLIC_APP_URL=http://localhost:3000
OTP_SECRET=d3c8e48db5d4d7f914b055368210266d69deb9b49c745247e1bdce39dbd2f760
```

1.4 Create .gitignore additions

Append to `.gitignore`:

```
.env.local
```

```
.env.production.local
*.pem
JMD_AUTOBUILD_PROMPT.md
```

1.5 Initialize git

```
git init
git add .
git commit -m "chore: initial JMD Online Book setup"
```

PHASE 2 — DATABASE SCHEMA

Open Supabase dashboard → <https://zkvrlwqcfeecsecrzlnu.supabase.co> → SQL Editor. Run this entire SQL block:

```
-- =====
-- JMD ONLINE BOOK — FULL PRODUCTION SCHEMA
-- Run once in Supabase SQL Editor
-- =====

CREATE EXTENSION IF NOT EXISTS "uuid-osspl";

-- — PROFILES —————
CREATE TABLE IF NOT EXISTS public.profiles (
  id          UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  phone       VARCHAR(20) UNIQUE,
  email       VARCHAR(200),
  full_name   VARCHAR(100),
  role        VARCHAR(20) DEFAULT 'user' CHECK (role IN ('user', 'agent', 'admi
  balance     DECIMAL(12,2) DEFAULT 0.00,
  bonus_balance DECIMAL(12,2) DEFAULT 0.00,
  total_deposited DECIMAL(12,2) DEFAULT 0.00,
  total_withdrawn DECIMAL(12,2) DEFAULT 0.00,
  total_won    DECIMAL(12,2) DEFAULT 0.00,
  total_lost   DECIMAL(12,2) DEFAULT 0.00,
  referral_code VARCHAR(20) UNIQUE DEFAULT UPPER(SUBSTR(MD5(RANDOM())::TEXT), 1,
  referred_by  UUID REFERENCES public.profiles(id),
  agent_id     UUID REFERENCES public.profiles(id),
  is_active    BOOLEAN DEFAULT TRUE,
  is_verified  BOOLEAN DEFAULT FALSE,
  avatar_url   TEXT,
```

```

    bank_account    VARCHAR(50),
    ifsc_code       VARCHAR(20),
    account_holder  VARCHAR(100),
    upi_id          VARCHAR(100),
    last_login_at   TIMESTAMPTZ,
    created_at      TIMESTAMPTZ DEFAULT NOW(),
    updated_at      TIMESTAMPTZ DEFAULT NOW()
);

```

```

-- — TRANSACTIONS —————

```

```

CREATE TABLE IF NOT EXISTS public.transactions (
    id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id           UUID NOT NULL REFERENCES public.profiles(id),
    type              VARCHAR(20) NOT NULL CHECK (type IN ('deposit','withdraw','bet
amount              DECIMAL(12,2) NOT NULL,
    balance_before    DECIMAL(12,2),
    balance_after     DECIMAL(12,2),
    status             VARCHAR(20) DEFAULT 'pending' CHECK (status IN ('pending','app
payment_method       VARCHAR(50),
    payment_reference VARCHAR(100),
    screenshot_url    TEXT,
    upi_id             VARCHAR(100),
    bank_account       VARCHAR(50),
    ifsc_code          VARCHAR(20),
    account_holder    VARCHAR(100),
    admin_note         TEXT,
    approved_by        UUID REFERENCES public.profiles(id),
    approved_at        TIMESTAMPTZ,
    created_at         TIMESTAMPTZ DEFAULT NOW(),
    updated_at         TIMESTAMPTZ DEFAULT NOW()
);

```

```

-- — GAMES —————

```

```

CREATE TABLE IF NOT EXISTS public.games (
    id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name              VARCHAR(100) NOT NULL,
    provider           VARCHAR(100) NOT NULL,
    category           VARCHAR(50) NOT NULL CHECK (category IN ('sports','casino','lotter
thumbnail_url        TEXT,
    launch_url         TEXT,
    description         TEXT,
    is_active           BOOLEAN DEFAULT TRUE,
    is_featured         BOOLEAN DEFAULT FALSE,
    is_hot              BOOLEAN DEFAULT FALSE,
    is_new              BOOLEAN DEFAULT FALSE,
    sort_order          INT DEFAULT 0,
    min_bet             DECIMAL(10,2) DEFAULT 10,

```

```
max_bet      DECIMAL(10,2) DEFAULT 100000,
tags         TEXT[] DEFAULT '{}',
play_count   INT DEFAULT 0,
created_at   TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- — BETS —————
```

```
CREATE TABLE IF NOT EXISTS public.bets (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id     UUID NOT NULL REFERENCES public.profiles(id),
  game_id     UUID NOT NULL REFERENCES public.games(id),
  amount      DECIMAL(12,2) NOT NULL,
  odds        DECIMAL(8,4),
  potential_win DECIMAL(12,2),
  result      VARCHAR(20) DEFAULT 'pending' CHECK (result IN ('pending','win',
  payout      DECIMAL(12,2),
  settled_at   TIMESTAMPTZ,
  metadata     JSONB DEFAULT '{}',
  created_at   TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- — COMMISSIONS —————
```

```
CREATE TABLE IF NOT EXISTS public.commissions (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  agent_id    UUID NOT NULL REFERENCES public.profiles(id),
  player_id   UUID NOT NULL REFERENCES public.profiles(id),
  transaction_id UUID REFERENCES public.transactions(id),
  amount      DECIMAL(12,2) NOT NULL,
  rate        DECIMAL(5,4) DEFAULT 0.05,
  type        VARCHAR(20) DEFAULT 'deposit',
  is_paid     BOOLEAN DEFAULT FALSE,
  created_at   TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- — NOTIFICATIONS —————
```

```
CREATE TABLE IF NOT EXISTS public.notifications (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id     UUID NOT NULL REFERENCES public.profiles(id),
  title       VARCHAR(200) NOT NULL,
  body        TEXT NOT NULL,
  type        VARCHAR(50) DEFAULT 'info' CHECK (type IN ('info','success','warning
  is_read     BOOLEAN DEFAULT FALSE,
  action_url  TEXT,
  metadata     JSONB DEFAULT '{}',
  created_at   TIMESTAMPTZ DEFAULT NOW()
);
```

```

-- — OTP TOKENS —————
CREATE TABLE IF NOT EXISTS public.otp_tokens (
    id            UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    identifier    VARCHAR(100) NOT NULL,
    token        VARCHAR(6) NOT NULL,
    purpose      VARCHAR(20) DEFAULT 'login' CHECK (purpose IN ('login','register','
    expires_at   TIMESTAMPTZ NOT NULL,
    is_used      BOOLEAN DEFAULT FALSE,
    attempt_count INT DEFAULT 0,
    created_at   TIMESTAMPTZ DEFAULT NOW()
);

-- — PAYMENT METHODS —————
CREATE TABLE IF NOT EXISTS public.payment_methods (
    id            UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name         VARCHAR(100) NOT NULL,
    type         VARCHAR(50) NOT NULL CHECK (type IN ('upi','bank','wallet','crypto'
    details      JSONB DEFAULT '{}',
    is_active    BOOLEAN DEFAULT TRUE,
    for_deposit  BOOLEAN DEFAULT TRUE,
    for_withdraw BOOLEAN DEFAULT TRUE,
    sort_order   INT DEFAULT 0,
    min_amount   DECIMAL(10,2) DEFAULT 100,
    max_amount   DECIMAL(10,2) DEFAULT 100000,
    created_at   TIMESTAMPTZ DEFAULT NOW()
);

-- — SITE SETTINGS —————
CREATE TABLE IF NOT EXISTS public.site_settings (
    key          VARCHAR(100) PRIMARY KEY,
    value        TEXT,
    type         VARCHAR(20) DEFAULT 'string' CHECK (type IN ('string','number','boo
    description   TEXT,
    updated_at   TIMESTAMPTZ DEFAULT NOW()
);

-- — INDEXES —————
CREATE INDEX IF NOT EXISTS idx_transactions_user_id      ON public.transactions(us
CREATE INDEX IF NOT EXISTS idx_transactions_status      ON public.transactions(st
CREATE INDEX IF NOT EXISTS idx_transactions_type        ON public.transactions(ty
CREATE INDEX IF NOT EXISTS idx_transactions_created_at  ON public.transactions(cr
CREATE INDEX IF NOT EXISTS idx_bets_user_id             ON public.bets(user_id);
CREATE INDEX IF NOT EXISTS idx_bets_game_id            ON public.bets(game_id);
CREATE INDEX IF NOT EXISTS idx_notifications_user_id    ON public.notifications(u
CREATE INDEX IF NOT EXISTS idx_otp_identifier          ON public.otp_tokens(iden
CREATE INDEX IF NOT EXISTS idx_profiles_referral_code  ON public.profiles(referr
CREATE INDEX IF NOT EXISTS idx_profiles_referred_by    ON public.profiles(referr

```

```

-- — ROW LEVEL SECURITY —————
ALTER TABLE public.profiles          ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.transactions      ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.bets              ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.notifications     ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.commissions       ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.games             ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.payment_methods   ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.site_settings     ENABLE ROW LEVEL SECURITY;

-- profiles policies
DROP POLICY IF EXISTS "own_profile_select" ON public.profiles;
DROP POLICY IF EXISTS "own_profile_update" ON public.profiles;
DROP POLICY IF EXISTS "profile_insert"     ON public.profiles;
DROP POLICY IF EXISTS "admin_all_profiles" ON public.profiles;

CREATE POLICY "own_profile_select" ON public.profiles FOR SELECT USING (auth.uid
CREATE POLICY "own_profile_update" ON public.profiles FOR UPDATE USING (auth.uid
CREATE POLICY "profile_insert"     ON public.profiles FOR INSERT WITH CHECK (aut
CREATE POLICY "admin_all_profiles" ON public.profiles FOR ALL
    USING (EXISTS (SELECT 1 FROM public.profiles p WHERE p.id = auth.uid() AND p.ro

-- transactions policies
CREATE POLICY "own_transactions_select" ON public.transactions FOR SELECT USING (
CREATE POLICY "own_transactions_insert" ON public.transactions FOR INSERT WITH CH
CREATE POLICY "admin_all_transactions"  ON public.transactions FOR ALL
    USING (EXISTS (SELECT 1 FROM public.profiles p WHERE p.id = auth.uid() AND p.ro

-- bets policies
CREATE POLICY "own_bets_select" ON public.bets FOR SELECT USING (auth.uid() = use
CREATE POLICY "own_bets_insert" ON public.bets FOR INSERT WITH CHECK (auth.uid()
CREATE POLICY "admin_bets"      ON public.bets FOR ALL
    USING (EXISTS (SELECT 1 FROM public.profiles p WHERE p.id = auth.uid() AND p.ro

-- notifications policies
CREATE POLICY "own_notifications" ON public.notifications FOR ALL USING (auth.uid

-- commissions policies
CREATE POLICY "own_commissions" ON public.commissions FOR SELECT USING (auth.uid(
CREATE POLICY "admin_commissions" ON public.commissions FOR ALL
    USING (EXISTS (SELECT 1 FROM public.profiles p WHERE p.id = auth.uid() AND p.ro

-- games: public read
CREATE POLICY "public_games_read" ON public.games FOR SELECT USING (is_active = t
CREATE POLICY "admin_games_all"   ON public.games FOR ALL
    USING (EXISTS (SELECT 1 FROM public.profiles p WHERE p.id = auth.uid() AND p.ro

```

```

-- payment_methods: public read active
CREATE POLICY "public_payment_methods" ON public.payment_methods FOR SELECT USING
CREATE POLICY "admin_payment_methods" ON public.payment_methods FOR ALL
    USING (EXISTS (SELECT 1 FROM public.profiles p WHERE p.id = auth.uid() AND p.ro

-- site_settings: public read
CREATE POLICY "public_settings_read" ON public.site_settings FOR SELECT USING (tr
CREATE POLICY "admin_settings_all" ON public.site_settings FOR ALL
    USING (EXISTS (SELECT 1 FROM public.profiles p WHERE p.id = auth.uid() AND p.ro

-- — REALTIME —————
ALTER PUBLICATION supabase_realtime ADD TABLE public.notifications;
ALTER PUBLICATION supabase_realtime ADD TABLE public.transactions;
ALTER PUBLICATION supabase_realtime ADD TABLE public.profiles;

-- — FUNCTIONS —————

-- Auto-create profile on signup
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER AS $$
BEGIN
    INSERT INTO public.profiles (id, full_name, email)
    VALUES (
        NEW.id,
        COALESCE(NEW.raw_user_meta_data->>'full_name', 'User'),
        NEW.email
    );
    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS on_auth_user_created ON auth.users;
CREATE TRIGGER on_auth_user_created
    AFTER INSERT ON auth.users
    FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();

-- Atomic balance update (prevents race conditions)
CREATE OR REPLACE FUNCTION public.update_balance(
    p_user_id UUID,
    p_amount DECIMAL,
    p_type TEXT
) RETURNS JSONB LANGUAGE plpgsql SECURITY DEFINER AS $$
DECLARE
    v_current DECIMAL;
    v_new DECIMAL;
BEGIN

```

```

SELECT balance INTO v_current FROM public.profiles WHERE id = p_user_id FOR UPD
IF v_current IS NULL THEN
    RETURN jsonb_build_object('success', false, 'error', 'User not found');
END IF;
IF p_type IN ('withdraw','bet') AND v_current < ABS(p_amount) THEN
    RETURN jsonb_build_object('success', false, 'error', 'Insufficient balance');
END IF;
v_new := v_current + p_amount;
UPDATE public.profiles
SET balance = v_new,
    total_deposited = CASE WHEN p_type = 'deposit' THEN total_deposited + ABS(p
    total_withdrawn = CASE WHEN p_type = 'withdraw' THEN total_withdrawn + ABS(
    updated_at = NOW()
WHERE id = p_user_id;
RETURN jsonb_build_object('success', true, 'balance', v_new, 'previous', v_curr
END;
$$;

```

-- Admin dashboard stats

```

CREATE OR REPLACE FUNCTION public.get_dashboard_stats()
RETURNS JSONB LANGUAGE plpgsql SECURITY DEFINER AS $$
DECLARE v JSONB;
BEGIN
    SELECT jsonb_build_object(
        'total_users',      (SELECT COUNT(*) FROM public.profiles WHERE role = 'user
        'total_agents',     (SELECT COUNT(*) FROM public.profiles WHERE role = 'agen
        'today_deposits',    COALESCE((SELECT SUM(amount) FROM public.transactions WH
        'today_withdrawals', COALESCE((SELECT SUM(amount) FROM public.transactions WH
        'pending_deposits',  (SELECT COUNT(*) FROM public.transactions WHERE type='de
        'pending_withdrawals', (SELECT COUNT(*) FROM public.transactions WHERE type='w
        'total_balance',     COALESCE((SELECT SUM(balance) FROM public.profiles WHERE
        'new_users_today',   (SELECT COUNT(*) FROM public.profiles WHERE created_at::
    ) INTO v;
    RETURN v;
END;
$$;

```

-- — STORAGE BUCKETS —————

```

INSERT INTO storage.buckets (id, name, public, file_size_limit, allowed_mime_type
VALUES ('screenshots', 'screenshots', true, 5242880, ARRAY['image/jpeg','image/pn
ON CONFLICT (id) DO NOTHING;

```

```

DROP POLICY IF EXISTS "auth_upload_screenshots" ON storage.objects;
DROP POLICY IF EXISTS "public_read_screenshots" ON storage.objects;
CREATE POLICY "auth_upload_screenshots" ON storage.objects FOR INSERT
    WITH CHECK (auth.role() = 'authenticated' AND bucket_id = 'screenshots');
CREATE POLICY "public_read_screenshots" ON storage.objects FOR SELECT

```

```

        USING (bucket_id = 'screenshots');

-- — SEED DATA —————
INSERT INTO public.site_settings (key, value, type, description) VALUES
    ('site_name',          'JMD Online Book',          'string'
    ('min_deposit',        '100',                      'number'
    ('max_deposit',        '100000',                    'number'
    ('min_withdraw',       '200',                      'number'
    ('max_withdraw_daily', '50000',                    'number'
    ('referral_commission', '0.05',                  'number'
    ('maintenance_mode',   'false',                    'boolean'
    ('welcome_bonus',      '0',                        'number'
    ('upi_id',             'jmdonlinebook@upi',          'string'
    ('whatsapp_support',    '+919999999999',              'string'
    ('announcement',       'Welcome to JMD Online Book! Fastest withdrawals gua
    ('allow_registration',  'true',                      'boolean'
    ('company_name',       'JMD Online Book',            'string'
    ('min_bet',            '10',                        'number'
ON CONFLICT (key) DO NOTHING;

INSERT INTO public.payment_methods (name, type, details, sort_order, min_amount,
    ('UPI',                'upi',    '{"upi_id":"jmdonlinebook@upi","display":"Scan & Pay"
    ('PhonePe',            'upi',    '{"upi_id":"jmdonlinebook@ybl","display":"PhonePe UPI
    ('Google Pay',         'upi',    '{"upi_id":"jmdonlinebook@okaxis","display":"GPay"}',
    ('Paytm',              'upi',    '{"upi_id":"jmdonlinebook@paytm","display":"Paytm"}',
    ('Bank Transfer',      'bank',  '{"bank_name":"HDFC Bank","account":"1234567890","ifs
ON CONFLICT DO NOTHING;

INSERT INTO public.games (name, provider, category, is_featured, is_hot, is_new,
    ('IPL Cricket Bet',    'JMD Sports',    'sports',    true,    true,    false, 1, 10,
    ('Teen Patti Gold',    'JMD Casino',    'cards',     true,    true,    false, 2, 10,
    ('Andar Bahar',        'JMD Casino',    'cards',     true,    false,   false, 3, 10,
    ('Roulette',           'JMD Casino',    'casino',    false,   false,   false, 4, 10,
    ('Dragon Tiger',        'JMD Casino',    'casino',    false,   true,    false, 5, 10,
    ('Lottery 3D',          'JMD Luck',      'lottery',   false,   false,   true,  6, 10,
    ('Football Bet',        'JMD Sports',    'sports',    false,   false,   false, 7, 10,
    ('Horse Racing',        'JMD Sports',    'sports',    false,   false,   false, 8, 50,
    ('Color Prediction',    'JMD Games',     'other',     true,    true,    false, 9, 10,
    ('Aviator',            'JMD Games',     'casino',    true,    true,    true,  10, 10,
    ('Kabaddi Bet',         'JMD Sports',    'sports',    false,   true,    true,  11, 10,
    ('Poker Texas',         'JMD Casino',    'cards',     false,   false,   false, 12, 50,
ON CONFLICT DO NOTHING;

```

After running that SQL, verify in Table Editor that all tables exist.

PHASE 3 — FULL FILE TREE CREATION

Create this exact directory structure and every file listed:

```
mkdir -p \
  src/app/(auth\)/login \
  src/app/(auth\)/register \
  src/app/(auth\)/verify-otp \
  src/app/(auth\)/forgot-password \
  src/app/(main\)/home \
  src/app/(main\)/wallet \
  src/app/(main\)/deposit \
  src/app/(main\)/withdraw \
  src/app/(main\)/transactions \
  src/app/(main\)/games/[id\] \
  src/app/(main\)/profile \
  src/app/(main\)/referral \
  src/app/(main\)/notifications \
  src/app/(admin\)/dashboard \
  src/app/(admin\)/users/[id\] \
  src/app/(admin\)/transactions/[id\] \
  src/app/(admin\)/games \
  src/app/(admin\)/payments \
  src/app/(admin\)/settings \
  src/app/api/auth/send-otp \
  src/app/api/auth/verify-otp \
  src/app/api/auth/register \
  src/app/api/wallet/deposit \
  src/app/api/wallet/withdraw \
  src/app/api/wallet/upload-screenshot \
  src/app/api/admin/transactions/approve \
  src/app/api/admin/users \
  src/app/api/admin/stats \
  src/app/api/referral/tree \
  src/components/ui \
  src/components/layout \
  src/components/wallet \
  src/components/games \
  src/components/admin \
  src/lib/supabase \
  src/lib/email \
  src/hooks \
  src/store \
  src/types \
  public/icons
```

PHASE 4 — WRITE ALL SOURCE FILES

Write every file below completely. No partial implementations.

4.1 TYPES

src/types/index.ts

Define these TypeScript types fully:

```
export type UserRole = 'user' | 'agent' | 'admin'
export type TransactionType = 'deposit' | 'withdraw' | 'bet' | 'win' | 'bonus' |
export type TransactionStatus = 'pending' | 'approved' | 'rejected' | 'completed'
export type GameCategory = 'sports' | 'casino' | 'lottery' | 'cards' | 'other'
export type BetResult = 'pending' | 'win' | 'lose' | 'draw' | 'void'
export type NotificationType = 'info' | 'success' | 'warning' | 'danger' | 'depos
export type PaymentMethodType = 'upi' | 'bank' | 'wallet' | 'crypto'

export interface Profile {
  id: string
  phone: string | null
  email: string | null
  full_name: string | null
  role: UserRole
  balance: number
  bonus_balance: number
  total_deposited: number
  total_withdrawn: number
  total_won: number
  total_lost: number
  referral_code: string
  referred_by: string | null
  agent_id: string | null
  is_active: boolean
  is_verified: boolean
  avatar_url: string | null
  bank_account: string | null
  ifsc_code: string | null
  account_holder: string | null
  upi_id: string | null
  last_login_at: string | null
  created_at: string
```

```
    updated_at: string
}
```

```
export interface Transaction {
  id: string
  user_id: string
  type: TransactionType
  amount: number
  balance_before: number | null
  balance_after: number | null
  status: TransactionStatus
  payment_method: string | null
  payment_reference: string | null
  screenshot_url: string | null
  upi_id: string | null
  bank_account: string | null
  ifsc_code: string | null
  account_holder: string | null
  admin_note: string | null
  approved_by: string | null
  approved_at: string | null
  created_at: string
  updated_at: string
  profiles?: Pick<Profile, 'full_name' | 'phone' | 'email'>
}
```

```
export interface Game {
  id: string
  name: string
  provider: string
  category: GameCategory
  thumbnail_url: string | null
  launch_url: string | null
  description: string | null
  is_active: boolean
  is_featured: boolean
  is_hot: boolean
  is_new: boolean
  sort_order: number
  min_bet: number
  max_bet: number
  tags: string[]
  play_count: number
  created_at: string
}
```

```
export interface Notification {
```

```
    id: string
    user_id: string
    title: string
    body: string
    type: NotificationType
    is_read: boolean
    action_url: string | null
    metadata: Record<string, unknown>
    created_at: string
}
```

```
export interface PaymentMethod {
    id: string
    name: string
    type: PaymentMethodType
    details: Record<string, string>
    is_active: boolean
    for_deposit: boolean
    for_withdraw: boolean
    sort_order: number
    min_amount: number
    max_amount: number
}
```

```
export interface SiteSettings {
    site_name: string
    min_deposit: number
    max_deposit: number
    min_withdraw: number
    max_withdraw_daily: number
    referral_commission: number
    maintenance_mode: boolean
    welcome_bonus: number
    upi_id: string
    whatsapp_support: string
    announcement: string
    allow_registration: boolean
}
```

```
export interface ApiResponse<T = unknown> {
    data?: T
    error?: string
    message?: string
}
```

```
export interface PaginatedResponse<T> {
    data: T[]
}
```

```

    count: number
    page: number
    limit: number
    hasMore: boolean
  }

export interface DashboardStats {
  total_users: number
  total_agents: number
  today_deposits: number
  today_withdrawals: number
  pending_deposits: number
  pending_withdrawals: number
  total_balance: number
  new_users_today: number
}

```

4.2 SUPABASE CLIENTS

src/lib/supabase/client.ts

```

import { createBrowserClient } from '@supabase/ssr'

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  )
}

```

src/lib/supabase/server.ts

```

import { createServerClient } from '@supabase/ssr'
import { cookies } from 'next/headers'

export async function createClient() {
  const cookieStore = await cookies()
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {

```

```

        getAll: () => cookieStore.getAll(),
        setAll: (toSet) => toSet.forEach(({ name, value, options }) => cookieStor
    },
  }
)
}

```

src/lib/supabase/admin.ts

```

import { createClient } from '@supabase/supabase-js'

export function createAdminClient() {
  return createClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY!,
    { auth: { autoRefreshToken: false, persistSession: false } }
  )
}

```

4.3 UTILITIES

src/lib/utlils.ts

```

import { clsx, type ClassValue } from 'clsx'
import { twMerge } from 'tailwind-merge'

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}

export function formatCurrency(amount: number): string {
  return new Intl.NumberFormat('en-IN', {
    style: 'currency', currency: 'INR',
    minimumFractionDigits: 2,
  }).format(amount)
}

export function formatDate(dateStr: string): string {
  return new Intl.DateTimeFormat('en-IN', {
    day: '2-digit', month: 'short', year: 'numeric',
    hour: '2-digit', minute: '2-digit',
  }).format(new Date(dateStr))
}

```

```

}

export function generateOTP(): string {
  return Math.floor(100000 + Math.random() * 900000).toString()
}

export function maskPhone(phone: string): string {
  if (!phone) return ''
  return phone.replace(/(\d{2})\d{6}(\d{2})/, '$1xxxxxx$2')
}

export function maskEmail(email: string): string {
  if (!email) return ''
  const [user, domain] = email.split('@')
  return `${user.slice(0, 2)}***@${domain}`
}

export function getTransactionColor(type: string): string {
  const colors: Record<string, string> = {
    deposit: 'text-emerald-400',
    win: 'text-emerald-400',
    bonus: 'text-emerald-400',
    referral: 'text-emerald-400',
    commission: 'text-emerald-400',
    withdraw: 'text-red-400',
    bet: 'text-red-400',
    adjustment: 'text-amber-400',
  }
  return colors[type] ?? 'text-gray-400'
}

export function getTransactionPrefix(type: string): string {
  return ['deposit', 'win', 'bonus', 'referral', 'commission'].includes(type) ? '+' :
}

export function getStatusBadgeClass(status: string): string {
  const classes: Record<string, string> = {
    pending: 'bg-yellow-500/15 text-yellow-400 border-yellow-500/30',
    approved: 'bg-emerald-500/15 text-emerald-400 border-emerald-500/30',
    completed: 'bg-emerald-500/15 text-emerald-400 border-emerald-500/30',
    rejected: 'bg-red-500/15 text-red-400 border-red-500/30',
    processing: 'bg-blue-500/15 text-blue-400 border-blue-500/30',
  }
  return classes[status] ?? 'bg-gray-500/15 text-gray-400'
}

export function sleep(ms: number) {

```



```

        </div>
      `
    },
  })

  if (error) return { success: false, error: error.message }
  return { success: true }
}

export async function verifyOTP(identifier: string, token: string): Promise<{ val
  const supabase = createAdminClient()

  const { data } = await supabase
    .from('otp_tokens')
    .select('*')
    .eq('identifier', identifier)
    .eq('token', token)
    .eq('is_used', false)
    .gt('expires_at', new Date().toISOString())
    .order('created_at', { ascending: false })
    .limit(1)
    .single()

  if (!data) return { valid: false, error: 'Invalid or expired OTP' }

  // Increment attempt count, mark used
  await supabase.from('otp_tokens').update({ is_used: true, attempt_count: (data.

  return { valid: true }
}

```

4.4 ZUSTAND STORES

src/store/useAuthStore.ts

```

import { create } from 'zustand'
import { persist } from 'zustand/middleware'
import type { Profile } from '@types'

interface AuthState {
  profile: Profile | null
  isLoading: boolean
  setProfile: (profile: Profile | null) => void
  updateBalance: (balance: number) => void

```

```

    updateProfile: (updates: Partial<Profile>) => void
    clearProfile: () => void
  }

export const useAuthStore = create<AuthState>()(
  persist(
    (set) => ({
      profile: null,
      isLoading: false,
      setProfile: (profile) => set({ profile }),
      updateBalance: (balance) => set((state) => ({
        profile: state.profile ? { ...state.profile, balance } : null,
      })),
      updateProfile: (updates) => set((state) => ({
        profile: state.profile ? { ...state.profile, ...updates } : null,
      })),
      clearProfile: () => set({ profile: null }),
    }),
    { name: 'jmd-auth', partialize: (s) => ({ profile: s.profile }) }
  )
)

```

src/store/useNotificationStore.ts

```

import { create } from 'zustand'
import type { Notification } from '@types'

interface NotificationState {
  notifications: Notification[]
  unreadCount: number
  setNotifications: (n: Notification[]) => void
  addNotification: (n: Notification) => void
  markRead: (id: string) => void
  markAllRead: () => void
}

export const useNotificationStore = create<NotificationState>((set) => ({
  notifications: [],
  unreadCount: 0,
  setNotifications: (notifications) =>
    set({ notifications, unreadCount: notifications.filter((n) => !n.is_read).length },
  addNotification: (notification) =>
    set((state) => ({
      notifications: [notification, ...state.notifications],
      unreadCount: state.unreadCount + (notification.is_read ? 0 : 1),
    })),

```

```

markRead: (id) =>
  set((state) => ({
    notifications: state.notifications.map((n) => n.id === id ? { ...n, is_read
    unreadCount: Math.max(0, state.unreadCount - 1),
  })),
markAllRead: () =>
  set((state) => ({
    notifications: state.notifications.map((n) => ({ ...n, is_read: true })),
    unreadCount: 0,
  })),
}))

```

src/store/useAppStore.ts

```

import { create } from 'zustand'
import type { SiteSettings, PaymentMethod } from '@types'

interface AppState {
  settings: Partial<SiteSettings>
  paymentMethods: PaymentMethod[]
  isMaintenanceMode: boolean
  setSettings: (s: Partial<SiteSettings>) => void
  setPaymentMethods: (m: PaymentMethod[]) => void
}

export const useAppStore = create<AppState>((set) => ({
  settings: {},
  paymentMethods: [],
  isMaintenanceMode: false,
  setSettings: (settings) =>
    set({ settings, isMaintenanceMode: settings.maintenance_mode === true }),
  setPaymentMethods: (paymentMethods) => set({ paymentMethods }),
}))

```

4.5 MIDDLEWARE

src/middleware.ts

```

import { createServerClient } from '@supabase/ssr'
import { NextResponse, type NextRequest } from 'next/server'

const PUBLIC = ['/login', '/register', '/verify-otp', '/forgot-password']

```

```

const ADMIN    = ['/admin']

export async function middleware(request: NextRequest) {
  let response = NextResponse.next({ request })

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => request.cookies.getAll(),
        setAll: (toSet) => toSet.forEach(({ name, value, options }) => response.c
      },
    }
  )

  const { data: { user } } = await supabase.auth.getUser()
  const { pathname } = request.nextUrl

  if (pathname === '/') {
    return NextResponse.redirect(new URL(user ? '/home' : '/login', request.url))
  }

  if (!user && !PUBLIC.some(p => pathname.startsWith(p))) {
    const loginUrl = new URL('/login', request.url)
    loginUrl.searchParams.set('redirect', pathname)
    return NextResponse.redirect(loginUrl)
  }

  if (user && PUBLIC.some(p => pathname.startsWith(p))) {
    return NextResponse.redirect(new URL('/home', request.url))
  }

  if (user && ADMIN.some(p => pathname.startsWith(p))) {
    const { data: profile } = await supabase
      .from('profiles').select('role').eq('id', user.id).single()
    if (!profile || ![ 'admin', 'agent' ].includes(profile.role)) {
      return NextResponse.redirect(new URL('/home', request.url))
    }
  }

  return response
}

export const config = {
  matcher: ['/(?!_next/static|_next/image|favicon.ico|icons|.*\\.(?:svg|png|jpg|
}

```

4.6 GLOBAL CSS & THEME

src/app/globals.css

```
@import "tailwindcss";

:root {
  --gold:      #f59e0b;
  --gold-light: #fbbf24;
  --gold-dark:  #d97706;
  --accent:     #7c3aed;
  --bg:         #070b13;
  --bg-card:    rgba(255,255,255,0.04);
  --bg-card-2:  rgba(255,255,255,0.07);
  --border:     rgba(255,255,255,0.08);
  --border-gold: rgba(245,158,11,0.3);
}

*, *::before, *::after { box-sizing: border-box; }
* { -webkit-tap-highlight-color: transparent; }

html { scroll-behavior: smooth; }
body {
  background: var(--bg);
  color: #f1f5f9;
  font-family: system-ui, -apple-system, sans-serif;
  overscroll-behavior-y: none;
  -webkit-font-smoothing: antialiased;
}

/* Hide number input arrows */
input[type="number"]::-webkit-outer-spin-button,
input[type="number"]::-webkit-inner-spin-button { -webkit-appearance: none; margin: 0; }
input[type="number"] { -moz-appearance: textfield; }

/* Scrollbar */
::-webkit-scrollbar { width: 3px; height: 3px; }
::-webkit-scrollbar-track { background: transparent; }
::-webkit-scrollbar-thumb { background: var(--gold-dark); border-radius: 10px; }

/* Safe area */
.pb-safe { padding-bottom: max(env(safe-area-inset-bottom), 0.5rem); }
```

```

/* Marquee */
@keyframes marquee { 0% { transform: translateX(100vw) } 100% { transform: transla
.marquee-text { animation: marquee 22s linear infinite; white-space: nowrap; }
.marquee-text:hover { animation-play-state: paused; }

/* Skeleton shimmer */
@keyframes shimmer { 0% { background-position: -200% 0 } 100% { background-positi
.skeleton {
  background: linear-gradient(90deg, rgba(255,255,255,0.04) 25%, rgba(255,255,255
  background-size: 200% 100%;
  animation: shimmer 1.5s infinite;
  border-radius: 8px;
}

/* Glass card */
.glass {
  background: var(--bg-card);
  backdrop-filter: blur(20px) saturate(150%);
  -webkit-backdrop-filter: blur(20px) saturate(150%);
  border: 1px solid var(--border);
  border-radius: 16px;
}

/* Gold gradient button */
.btn-gold {
  background: linear-gradient(135deg, #f59e0b 0%, #d97706 100%);
  color: #000;
  font-weight: 700;
  border-radius: 12px;
  transition: transform 0.12s, box-shadow 0.12s;
  box-shadow: 0 4px 20px rgba(245,158,11,0.3);
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 8px;
  cursor: pointer;
  border: none;
  width: 100%;
  padding: 14px 24px;
  font-size: 16px;
}
.btn-gold:active { transform: scale(0.97); box-shadow: 0 2px 10px rgba(245,158,11
.btn-gold:disabled { opacity: 0.5; cursor: not-allowed; transform: none; }

/* Dark input */
.input-field {
  width: 100%;

```

```

background: rgba(255,255,255,0.06);
border: 1px solid var(--border);
border-radius: 12px;
padding: 14px 16px;
color: #fff;
font-size: 15px;
outline: none;
transition: border-color 0.2s, box-shadow 0.2s;
-webkit-appearance: none;
}
.input-field::placeholder { color: #4b5563; }
.input-field:focus {
  border-color: var(--gold);
  box-shadow: 0 0 0 3px rgba(245,158,11,0.12);
}
.input-field.error { border-color: #ef4444; }

```

4.7 ROOT LAYOUT

src/app/layout.tsx

```

import type { Metadata, Viewport } from 'next'
import './globals.css'
import { Toaster } from 'react-hot-toast'

export const metadata: Metadata = {
  title: 'JMD Online Book',
  description: 'Your most trusted online gaming platform. Fast withdrawals guaran',
  manifest: '/manifest.json',
  appleWebApp: { capable: true, statusBarStyle: 'black-translucent', title: 'JMD' },
  icons: { apple: '/icons/icon-192.png' },
}

export const viewport: Viewport = {
  width: 'device-width',
  initialScale: 1,
  maximumScale: 1,
  userScalable: false,
  themeColor: '#070b13',
}

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (

```

```

<html lang="en" suppressHydrationWarning>
  <body>
    {children}
    <Toaster
      position="top-center"
      toastOptions={{
        duration: 3500,
        style: { background: '#131b2b', color: '#fff', border: '1px solid rgb
        success: { iconTheme: { primary: '#f59e0b', secondary: '#000' } },
      }}
    />
  </body>
</html>
)
}

```

4.8 AUTH PAGES (complete implementations)

src/app/(auth)/login/page.tsx

Full login page with:

- Phone number input (auto-strips leading 0, adds +91 display)
- Password field with eye toggle
- “Login with OTP” tab — switches form to show email input, sends OTP via `/api/auth/send-otp`, **redirects to** `/verify-otp?email=xxx&redirect=/home`
- Form validation with react-hook-form + Zod: phone required 10 digits, password min 6 chars
- On submit: call `supabase.auth.signInWithPassword({ phone/email, password })`
- On success: fetch profile from `profiles` table, setProfile in store, redirect `/home`
- Error messages shown inline under fields
- Loading state on submit button
- Link to `/register` and `/forgot-password`
- Bottom: WhatsApp support link (opens `wa.me/+919999999999`)

src/app/(auth)/register/page.tsx

Full registration page with:

- Full name, phone (10 digits, format +91XXXXXXXXXX for Supabase), email, password, confirm password, referral code (optional, pre-fill from URL `?ref=CODE`)
- Terms checkbox — must be checked to submit
- Zod schema: name min 2, phone exactly 10 digits, email valid format (optional), password min 8 chars with regex requiring letter+number, passwords match
- On submit: `supabase.auth.signUp({ email: derivedEmail, phone: formattedPhone, password, options: { data: { full_name } } })`
- After signup: if referral code provided, call server action to set `referred_by` on profile
- Auto sign in and redirect to `/home`
- Link back to `/login`

`src/app/(auth)/verify-otp/page.tsx`

OTP verification page with:

- Read `email` and `redirect` from URL params
- Display masked email/phone
- 6 individual `<input maxLength={1}>` boxes in a flex row
 - Auto-focus next box on input
 - Backspace goes to previous box
 - Paste event fills all boxes at once
 - Only accept digits
- Auto-submit when all 6 filled
- On submit: POST to `/api/auth/verify-otp` , on success redirect to `redirect` param or `/home`
- Countdown timer: 60 seconds, then "Resend" button appears
- Resend calls `/api/auth/send-otp` again
- Error state with shake animation if wrong OTP

`src/app/(auth)/forgot-password/page.tsx`

Step 1: Enter email → send OTP via `/api/auth/send-otp` Step 2 (after OTP verified): New password + confirm → `supabase.auth.updateUser({ password })` Multi-step form with progress indicator.

4.9 MAIN LAYOUT

src/app/(main)/layout.tsx

Server component that:

- Creates supabase server client
- Gets user session — redirect to `/login` if none
- Fetches profile from `profiles` table
- Fetches `site_settings` rows
- Fetches active `payment_methods`
- Fetches unread notifications count
- Returns layout with `<RealtimeProvider>`, `<Header>`, `<main>`, `<MobileNav>`

src/components/layout/RealtimeProvider.tsx

Client component (wraps children):

- On mount: subscribe to `profiles` table changes filtered by user id → call `updateBalance` in store
- Subscribe to `notifications` table INSERT filtered by user id → call `addNotification` + `toast`
- Unsubscribe on unmount

src/components/layout/Header.tsx

```
// Shows: Logo (J icon + "JMD BOOK"), centered balance (₹X,XXX.XX from store),
// right side: notification bell with red badge count, support icon
// Balance has a subtle pulse animation when it changes
// Clicking bell → navigates to /notifications
// Clicking support → opens WhatsApp link
```

src/components/layout/MobileNav.tsx

```
// Fixed bottom navigation with 5 tabs:
// Home (/home), Wallet (/wallet), Games (/games), History (/transactions), Profi
// Active tab: amber color + scale 1.1 + filled icon
// Inactive: gray-500 + outline icon
// Include safe-area-inset-bottom padding
// Icons from react-icons/io5
// Labels below each icon (10px)
```

4.10 MAIN PAGES (complete implementations)

src/app/(main)/home/page.tsx

Build complete home page with these sections in order:

1. BALANCE CARD

- bg-gradient from amber-500 via-yellow-400 to-orange-400
- "Total Balance" label (small, 80% opacity)
- Balance amount formatted as ₹X,XXX.XX (from store, framer-motion number anim
- Two buttons row: "Deposit" (dark glass) and "Withdraw" (dark glass)
- Subtle gold glow shadow

2. ANNOUNCEMENT MARQUEE

- Dark glass card, amber left border
-  icon + scrolling text from site_settings.announcement
- Pause on hover

3. QUICK ACTIONS GRID (4 columns)

- Deposit (green gradient icon bg), Withdraw (red), Live Games (blue), Support
- Icon + label below
- Tap opens respective route or WhatsApp

4. FEATURED GAMES (horizontal scroll, no scrollbar)

- "🔥 Featured" heading + "See All" link
- Horizontal scrollable row of GameCard components (featured=true games)

5. CATEGORY TABS

- Pills: All, Sports, Casino, Cards, Lottery
- Filters the game grid below
- Selected tab: amber background

6. GAME GRID (3 columns)

- All active games matching selected category
- GameCard component for each

7. BOTTOM PROMO CARD

- "Refer & Earn" card with amber gradient
- Shows user's referral code
- "Share Now" button → navigator.share or copy to clipboard

src/app/(main)/wallet/page.tsx

Complete wallet page:

1. BALANCE CARD (purple gradient variant)
 - Available balance + bonus balance separately shown
 - Deposit + Withdraw buttons
2. QUICK STATS ROW (3 cards)
 - Total Deposited | Total Won | Total Withdrawn
 - Small label + formatted amount
3. RECENT TRANSACTIONS (last 10)
 - "Recent Transactions" heading + "View All" → /transactions
 - TransactionItem component for each
 - Pull-to-refresh (optional, use button)
 - Empty state if no transactions

src/app/(main)/deposit/page.tsx

Complete deposit flow:

STEP 1 — AMOUNT

- Input with ₹ prefix, number keyboard
- Quick chips: ₹100 ₹200 ₹500 ₹1K ₹2K ₹5K ₹10K
- "Min: ₹{min_deposit}" shown below
- Continue button

STEP 2 — METHOD

- Payment method cards (from paymentMethods in store, for_deposit=true)
- Each card: icon emoji + name + details preview
- Selected: amber border highlight

STEP 3 — PAY

- Show company payment details based on selected method:
 - UPI: Show UPI ID in large text + copy button + QR code placeholder

- Bank: Show full bank details with copy buttons for each field
- "I have made the payment" button → proceed to STEP 4

STEP 4 – CONFIRM

- Upload payment screenshot (required): image picker → preview → upload to Supaba
- UTR / Reference number input (required)
- Optional note
- Submit → POST /api/wallet/deposit
- Success screen: "Request Submitted! ✓ We'll verify within 30 minutes"
- Back to wallet button

src/app/(main)/withdraw/page.tsx

Complete withdrawal flow:

STEP 1 – AMOUNT

- Show available balance
- Amount input with quick chips
- "Min withdraw: ₹{min_withdraw}" warning
- Validate: amount <= balance, amount >= min_withdraw

STEP 2 – BANK DETAILS

- Toggle: UPI / Bank Transfer
- UPI mode: UPI ID input
- Bank mode: Account Number, IFSC Code, Account Holder Name
- "Save for future" checkbox – saves to profile if checked (PATCH /api/profile/ba
- If profile has saved details, pre-fill them with "Edit" option

STEP 3 – CONFIRM

- Summary: amount, method, account details
- "Confirm Withdrawal" button → POST /api/wallet/withdraw
- Success screen: "Withdrawal Requested! ✓ Processed within 24 hours"

src/app/(main)/transactions/page.tsx

Full transaction history:

1. FILTER BAR

- Type tabs: All | Deposit | Withdraw | Bet/Win
- Status pills: All | Pending | Completed | Rejected

2. TRANSACTION LIST

- Load 20 per page from Supabase (ordered by created_at DESC)
- Filter applied via Supabase query params

- TransactionItem for each (icon, type, amount colored, status badge, time)
- "Load More" button for pagination
- Empty state with illustration message

3. TRANSACTION DETAIL MODAL

- Tap any item → BottomSheet slides up
- Shows full details: ID, amount, method, reference, screenshot link, status,
- "Close" button

src/app/(main)/games/page.tsx

Game lobby:

1. SEARCH BAR (with filter icon)
2. CATEGORY TABS (scrollable: All, Sports, Casino, Cards, Lottery, Other)
3. FEATURED BANNER (top 1-2 featured games as full-width cards)
4. GAME GRID (3-column, filtered by category + search)
 - Each GameCard: thumbnail emoji (or img), name, provider, HOT/NEW badge, min
 - Tap → /games/[id]
5. EMPTY STATE if no games match filter

src/app/(main)/games/[id]/page.tsx

Game detail page:

- Game name, provider, category badge
- Thumbnail (large)
- Description
- Min/Max bet info
- "Play Now" button (links to launch_url if set, else shows "Coming Soon" toast)
- Recent bets by user in this game (from bets table)
- Back button

src/app/(main)/profile/page.tsx

Profile page:

1. AVATAR + NAME + PHONE + ROLE BADGE + VERIFIED BADGE (if is_verified)
2. STATS ROW
 - Balance | Total Deposited | Total Withdrawn
3. MENU LIST (each as a pressable row with icon + label + chevron):
 - Edit Profile → modal to edit name, email, avatar

- Bank Details → edit bank/UPI
- Change Password → triggers OTP then new password form
- Referral & Earnings → /referral
- Notifications → /notifications
- Logout → `supabase.auth.signOut()` + `clearProfile()` + `redirect /login`

4. ACCOUNT INFO (small gray text)

- Member since (`created_at` formatted)
- User ID (first 8 chars)
- Referral code with copy button

src/app/(main)/referral/page.tsx

Referral & Earnings page:

1. REFERRAL CODE DISPLAY

- Big bold code in glass card
- Copy button (copies full URL: `APP_URL/register?ref=CODE`)
- Share button (`navigator.share`)
- QR code (generate via URL: `api.qrserver.com/v1/create-qr-code/?size=180x180&`

2. STATS CARDS (3 grid)

- Total Referred | Active Players | Commission Earned

3. HOW IT WORKS

- 3 steps: Share code → Friend registers → Friend deposits → You earn 5%

4. COMMISSION HISTORY

- Load from `commissions` table where `agent_id = user.id`
- Each row: player name (masked), amount, date
- Empty state if none

5. MY REFERRALS LIST

- Load from `profiles` where `referred_by = user.id`
- Each: name (masked), date joined, total deposited

src/app/(main)/notifications/page.tsx

Notifications page:

- "Mark all read" button (top right)
- Grouped by date (Today, Yesterday, Earlier)
- Each item: colored icon by type, title, body, relative time
- Tap → mark as read + navigate to `action_url` if set
- Empty state: "No notifications yet"

- Unread items have subtle amber left border
-

4.11 ADMIN PAGES (complete implementations)

`src/app/(admin)/layout.tsx`

Admin layout with:

- Desktop: collapsible sidebar + main content
- Mobile: bottom admin nav (Dashboard, Txns, Users, Settings)
- Sidebar items: Dashboard, Transactions, Users, Games, Payments, Settings
- Logout button at bottom
- Role check (admin only, agents see limited menu)

`src/app/(admin)/dashboard/page.tsx`

Admin dashboard:

1. STATS GRID (2x4 on mobile, 4-col on desktop)
 Fetched from `/api/admin/stats`:
 - Total Users, Today Deposits (₹), Today Withdrawals (₹), Pending Approvals (w.
 - Total Balance in System, Total Agents, New Users Today, All-time Deposits
2. PENDING ALERTS
 - If `pending_deposits > 0`: yellow alert card "X deposits pending approval"
 - If `pending_withdrawals > 0`: orange alert card "X withdrawals pending"
 - Click → navigates to `/admin/transactions`
3. CHARTS (Recharts)
 - Line chart: last 7 days deposit vs withdrawal amounts
 - Bar chart: daily new user registrations for last 7 days
4. RECENT TRANSACTIONS TABLE
 - Last 10 pending transactions
 - Columns: User, Type, Amount, Method, Reference, Time, Actions
 - Quick Approve/Reject buttons inline

`src/app/(admin)/transactions/page.tsx`

Transaction management:

TABS: Pending Deposits | Pending Withdrawals | All Transactions

FOR PENDING TABS:

- Each item as a card (mobile) or table row (desktop):
 - User name + phone
 - Amount (large, colored)
 - Payment method + reference number
 - Screenshot link → opens in new tab or lightbox
 - Time ago
 -  Approve button → opens confirm modal → POST /api/admin/transactions/approv
 -  Reject button → opens modal with note textarea → POST {action:'reject', nc

ALL TRANSACTIONS TAB:

- Full paginated table
- Filters: type, status, date range, user search
- Export to CSV button (generate client-side from loaded data)

src/app/(admin)/users/page.tsx

User management:

- Search by name/phone
- Filter by role (user/agent/admin)
- Sortable table: Name, Phone, Balance, Total Dep, Total With, Role, Status, Join
- Tap row → /admin/users/[id]
- Bulk actions: (de)activate users

src/app/(admin)/users/[id]/page.tsx

User detail:

- Profile card with all details
- Balance adjustment form: amount + reason + type (credit/debit)
 - Uses /api/admin/users/[id]/adjust-balance
- Transaction history for this user
- Change role (user ↔ agent ↔ admin)
- Activate/deactivate toggle
- Referral tree: who they referred

src/app/(admin)/settings/page.tsx

Site settings editor:

- Load all from site_settings table
 - Each setting as a labeled input (correct type: text/number/toggle/textarea)
 - Save button per section or global save
 - Sections: General | Payments | Limits | Referral | Maintenance
 - Maintenance mode toggle – prominent warning card
-

4.12 API ROUTES (complete implementations)

src/app/api/auth/send-otp/route.ts

```
// POST { email: string, purpose?: string }
// Validate email with Zod
// Call sendOTPEmail from lib/email/otp.ts
// Return 200 { message: "OTP sent" } or 400/429 with error
```

src/app/api/auth/verify-otp/route.ts

```
// POST { identifier: string, token: string }
// Call verifyOTP from lib/email/otp.ts
// Return 200 { valid: true } or 400 { error: "Invalid OTP" }
```

src/app/api/wallet/deposit/route.ts

```
// POST – authenticated
// Validate: amount >= min_deposit, screenshot_url required, reference required
// Zod schema: { amount: z.number().min(100), payment_method: z.string(), payment
// Get user from supabase server client
// Check user is_active
// Get balance_before from profile
// Insert transaction { user_id, type:'deposit', amount, status:'pending', paymen
// Insert notification for admins (select all admins, insert notifications for ea
// Return { data: { transaction_id } }
```

src/app/api/wallet/withdraw/route.ts

```
// POST – authenticated
// Zod validate: { amount, bank_account?, ifsc_code?, account_holder?, upi_id?, p
// Get user profile – check balance >= amount, check amount >= min_withdraw
// Insert transaction { type:'withdraw', status:'pending', ... }
// If save_details: update profile bank fields
```

```
// Return { data: { transaction_id } }
```

src/app/api/wallet/upload-screenshot/route.ts

```
// POST multipart/form-data - authenticated
// Extract file from formData.get('file')
// Validate: file exists, size <= 5MB, type in ['image/jpeg','image/png','image/w
// Generate unique filename: {userId}/{Date.now()}-{random}.{ext}
// Upload to Supabase Storage bucket 'screenshots' using admin client
// Return { data: { url: publicUrl } }
```

src/app/api/admin/transactions/approve/route.ts

```
// POST - admin only (verify role from profiles)
// Zod: { transaction_id: z.string().uuid(), action: z.enum(['approve','reject'])
// Fetch transaction - verify it exists and status is 'pending'
// If approve:
//   Call update_balance DB function: amount for deposit, -amount for withdraw
//   If result.success = false: return 400 insufficient balance
//   Update transaction: status='approved', balance_before, balance_after, approv
//   If deposit and user has referred_by:
//     Get agent profile
//     Calculate commission = amount * referral_commission rate
//     Call update_balance for agent (credit commission)
//     Insert commission record
//     Insert notification for agent "Commission earned: ₹X"
//   Insert notification for user: "Deposit Approved: ₹X added to your account" (
//                                     or "Withdrawal Approved: ₹X processed" (withd
// If reject:
//   Update transaction: status='rejected', admin_note=note
//   Insert notification for user: "Your request was rejected. Reason: {note}"
// Return { data: { success: true } }
```

src/app/api/admin/stats/route.ts

```
// GET - admin only
// Call get_dashboard_stats() RPC
// Also fetch 7-day chart data:
//   SELECT date_trunc('day', created_at), SUM(amount) FROM transactions
//   WHERE created_at > NOW() - INTERVAL '7 days' AND status='approved'
//   GROUP BY 1, type
// Return combined stats object
```

src/app/api/referral/tree/route.ts

```
// GET - authenticated
// Fetch direct referrals: profiles where referred_by = user.id
// For each, fetch their deposit total from transactions
// Fetch total commission earned: SUM from commissions where agent_id = user.id
// Return { referrals, totalCommission, referralCode }
```

4.13 REUSABLE COMPONENTS (all fully implemented)

src/components/ui/Button.tsx

```
// Props: variant ('gold'|'outline'|'ghost'|'danger'), size ('sm'|'md'|'lg'),
//         loading (boolean), disabled, onClick, children, className, type
// Gold: btn-gold class
// Outline: border border-amber-500 text-amber-400 bg-transparent hover:bg-amber-
// Ghost: text-gray-400 hover:text-white hover:bg-white/5
// Danger: bg-red-500/10 text-red-400 border border-red-500/30 hover:bg-red-500/2
// Loading: show spinner (animate-spin), disable interactions
```

src/components/ui/Input.tsx

```
// Props: label, error, helperText, iconLeft, iconRight, ...HTMLInputAttributes
// Renders label above, input-field class, error message below in red
// When iconLeft provided: add pl-11, position icon absolutely left-3
```

src/components/ui/Modal.tsx

```
// Full screen overlay with backdrop blur
// Centered modal card with slide-up + fade-in (framer-motion)
// Close on backdrop click (optional, closeable prop)
// Props: isOpen, onClose, title, children, footer
```

src/components/ui/BottomSheet.tsx

```
// Slides up from bottom with handle bar
// Drag to dismiss (framer-motion drag)
// Backdrop overlay
// Props: isOpen, onClose, title, children
```

src/components/ui/Badge.tsx

```
// Status badge using getStatusBadgeClass from utils
// Also type variant for transaction types
// Small pill with border
```

src/components/ui/OTPInput.tsx

```
// 6 individual <input> boxes
// Auto-advance, backspace handling, paste support
// Props: value (string[6]), onChange, error
// On complete: call onComplete callback
```

src/components/ui/AmountChips.tsx

```
// Horizontal scrollable row of preset amount buttons
// Props: presets (number[]), selected (number|null), onSelect
// Selected chip: amber fill, others: glass outlined
```

src/components/ui/Loader.tsx

```
// Full page: centered gold spinning ring + "JMD" text
// Inline: small amber spinner
// Props: fullPage (boolean), size
```

src/components/ui/Skeleton.tsx

```
// Skeleton placeholder using .skeleton CSS class
// Props: width, height, className
// Compound: SkeletonCard, SkeletonList (renders N skeleton rows)
```

src/components/wallet/TransactionItem.tsx

```
// Props: transaction (Transaction type)
// Left: colored circle icon by type (↓ deposit green, ↑ withdraw red, 🎮 bet, 🏆
// Center: type label (capitalize) + "Ref: {reference}" if exists + date formatted
// Right: amount with +/- prefix, colored, status badge below
// Pressable (onClick prop)
```

src/components/games/GameCard.tsx

```
// Props: game (Game type), onClick
// Card with thumbnail (emoji fallback), name, provider text,
// HOT badge (red top-right) if is_hot, NEW badge (blue) if is_new
// Min bet shown at bottom
// Press animation: scale 0.96 on active
```

4.14 HOOKS

src/hooks/useProfile.ts

```
// Client-side hook that fetches and syncs profile
// Uses createClient() browser Supabase
// On mount: supabase.auth.getUser() → fetch profile → setProfile
// Sets up auth state listener
// Returns: { profile, isLoading, refetch }
```

src/hooks/useSupabase.ts

```
// Returns stable supabase browser client instance
// Created once with useMemo
```

src/hooks/useSettings.ts

```
// Fetches site_settings on mount
// Parses into SiteSettings object (convert type fields: numbers as parseFloat, b
// Calls setSettings in useAppStore
// Returns { settings, isLoading }
```

4.15 PWA

public/manifest.json

```
{
  "name": "JMD Online Book",
  "short_name": "JMD Book",
```

```

    "description": "Your trusted online gaming platform",
    "start_url": "/home",
    "display": "standalone",
    "background_color": "#070b13",
    "theme_color": "#f59e0b",
    "orientation": "portrait-primary",
    "categories": ["entertainment", "games"],
    "icons": [
      { "src": "/icons/icon-192.png", "sizes": "192x192", "type": "image/png", "pur
      { "src": "/icons/icon-512.png", "sizes": "512x512", "type": "image/png", "pur
    ]
  }
}

```

public/icons/ — generate these programmatically:

```

// Create a script: scripts/generate-icons.mjs
// Use canvas API (node-canvas) or just create a simple SVG-based PNG
// Icon: Gold rounded square, black "J" letter centered, 192x192 and 512x512
// npm install canvas then run: node scripts/generate-icons.mjs

```

4.16 NEXT.JS CONFIG

next.config.ts

```

import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  images: {
    remotePatterns: [
      { protocol: 'https', hostname: 'zkvrlwqcfeecsecrzlnu.supabase.co' },
      { protocol: 'https', hostname: 'api.qrserver.com' },
    ],
  },
  async headers() {
    return [
      {
        source: '/(.*)',
        headers: [
          { key: 'X-Frame-Options', value: 'SAMEORIGIN' },
          { key: 'X-Content-Type-Options', value: 'nosniff' },
          { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
          { key: 'Permissions-Policy', value: 'camera=(), microphone=(), geolocat

```

```
    ],  
    },  
  ],  
},  
}  
  
export default nextConfig
```

PHASE 5 — PRE-DEPLOY VERIFICATION

Run these and fix all errors before deploying:

```
# TypeScript check  
npx tsc --noEmit  
  
# Build check (MUST pass with 0 errors)  
npm run build  
  
# If build passes, commit everything  
git add -A  
git commit -m "feat: complete JMD Online Book production build"
```

Fix every TypeScript error and build warning before proceeding.

PHASE 6 — GITHUB + VERCEL AUTO-DEPLOY

6.1 Create GitHub repo

```
# Option A: GitHub CLI (recommended)  
gh auth login  
gh repo create jmd-online-book --private --push --source=.  
  
# Option B: Manual  
# Go to github.com → New repository → "jmd-online-book" → Private  
# git remote add origin https://github.com/YOUR_USERNAME/jmd-online-book.git  
# git push -u origin main
```

6.2 Deploy to Vercel (free)

```
# Install Vercel CLI
npm install -g vercel

# Login (opens browser)
vercel login

# Deploy (follow prompts: link to GitHub repo)
vercel --prod

# Vercel will ask:
# - Set up and deploy? Y
# - Which scope? (your account)
# - Link to existing project? N
# - Project name: jmd-online-book
# - Directory: ./
# - Override settings? N

# Note the deployment URL e.g. https://jmd-online-book.vercel.app
```

6.3 Set environment variables on Vercel

```
# Set all env vars via CLI (do not skip any)
vercel env add NEXT_PUBLIC_SUPABASE_URL production
# Enter: https://zkvrlwqcfeecsecrzlnu.supabase.co

vercel env add NEXT_PUBLIC_SUPABASE_ANON_KEY production
# Enter: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6Inp

vercel env add SUPABASE_SERVICE_ROLE_KEY production
# Enter: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6Inp

vercel env add RESEND_API_KEY production
# Enter: re_fHwpmKsC_2wXwZgxfssLHXBmDj36obxj2

vercel env add NEXT_PUBLIC_APP_NAME production
# Enter: JMD Online Book

vercel env add OTP_SECRET production
# Enter: d3c8e48db5d4d7f914b055368210266d69deb9b49c745247e1bdce39dbd2f760

# Set APP_URL LAST after you know the deployment URL
vercel env add NEXT_PUBLIC_APP_URL production
# Enter: https://jmd-online-book.vercel.app (or your actual URL)
```

6.4 Redeploy with env vars

```
vercel --prod
```

6.5 Verify Supabase Auth callback URL

In Supabase dashboard → Authentication → URL Configuration:

- Site URL: `https://jmd-online-book.vercel.app`
- Add to Redirect URLs: `https://jmd-online-book.vercel.app/**`

6.6 Enable GitHub auto-deploy

In Vercel dashboard → Project → Settings → Git:

- Connected to your GitHub repo
- Production branch: `main`
- Every `git push origin main` auto-deploys to production

PHASE 7 — ADMIN BOOTSTRAP

After first deployment, run in Supabase SQL Editor:

```
-- Create your admin account first: register normally on the live site
-- Then get your user ID and run:

-- Replace with your actual phone or email used to register
UPDATE public.profiles
SET role = 'admin', is_verified = true
WHERE phone = '91XXXXXXXXXX' -- or email = 'your@email.com'
RETURNING id, full_name, role;
```

Now log in and navigate to `/admin/dashboard`.

PHASE 8 — POST-DEPLOY CHECKLIST

Verify every item before considering the build complete:

Auth:

- Register new account → redirects to /home
- Login with credentials works
- OTP email received via Resend
- Logout clears session

User flows:

- Deposit form → screenshot uploads to Supabase Storage
- Transaction appears in /transactions as pending
- Admin /admin/transactions shows pending item
- Admin approves → user balance updates in realtime (without refresh)
- Toast notification appears on user's screen when balance changes
- Referral code visible in /referral
- Register with referral code → referred_by set on profile

Admin:

- /admin/dashboard loads stats
- Charts render without errors
- User table paginates
- Settings save correctly

Performance:

- `next build` → 0 errors, 0 warnings
- Vercel deployment status: Ready ✓
- Mobile Lighthouse score > 80

EXECUTION ORDER (strictly follow)

1. PHASE 1: Bootstrap + install

2. PHASE 2: Run SQL in Supabase (entire block at once)
 3. PHASE 3: Create directory structure
 4. PHASE 4: Write ALL files (4.1 → 4.16 in order)
 5. PHASE 5: TypeScript check + `npm run build` → fix all errors
 6. PHASE 6: GitHub + Vercel deploy
 7. PHASE 7: Admin bootstrap SQL
 8. PHASE 8: Verify checklist
-

STRICT RULES FOR AI AGENT

1. **Never write a stub.** Every function body must be fully implemented.
 2. **Never write** `// TODO` or `// implement later`. Do it now.
 3. **Never skip a file.** Every file in Phase 4 must be created.
 4. **Always use these exact env var names.** Never hardcode keys.
 5. **Mobile-first always.** All UI works perfectly at 375px.
 6. **Dark theme only.** Background `#070b13`, cards `rgba(255,255,255,0.04)`.
 7. **Gold accent always.** Primary CTA color is `#f59e0b`.
 8. **Fix TypeScript errors immediately** — don't accumulate them.
 9. **Handle all errors** — every API route returns proper status + `{ error }` on failure.
 10. **Zod validate every API input** — never trust raw request body.
 11. **Admin role verified server-side** on every admin route — never trust client claim.
 12. **Atomic balance updates** — always use `update_balance` RPC, never direct UPDATE.
 13. **Run** `npm run build` successfully before committing to git.
 14. **Report progress** after each phase completes.
-

BEGIN PHASE 1 NOW.

